Hybrid Exploration Controller for TurtleBot3

Mobile Robotics Project

Simulation and physical robot deployment

Adhithya Palanisamy Chandrasekaran

Umea University

January 25, 2026

Public portfolio copy

The title page was replaced to omit the university identifier.

The remaining report pages retain their submitted content.

Contents

1 Abstract **2**

2 Introduction **2**

3 System Overview and architecture **2**

3.1 Hybrid architecture 2

3.1.1 Deliberative components (planning / decision making) 3

3.1.2 Reactive Components 3

3.2 Nodes and topics flow 3

4 Algorithms **4**

4.1 Frontier extraction and clustering 4

4.2 Path Planning (A*) 6

4.3 Path Follower and collision avoidance 9

5 Results **9**

5.1 Simulation 10

5.2 Real Robot Deployment 10

6 Development experience and Challenges **11**

7 Conclusion **12**

1 Abstract

This project implements a **hybrid control system** for a TurtleBot3 that explores and maps a house-like environment. Exploration is split into **(i) a deliberative layer** that selects frontier goals and plans global paths on the map, and **(ii) a reactive layer** that uses the laser scanner directly for fast obstacle avoidance, progress monitoring, and re-planning triggers. Frontiers are detected as free cells adjacent to unknown space, clustered with an 8-connected flood-fill, and ranked by distance and cluster size. For each selected goal, an A* planner runs on an inflated occupancy grid with multiple goal candidates to improve robustness, and the produced path is smoothed before execution. The integrated system was validated in Gazebo and then adapted to a physical TurtleBot3, where sensor quality-of-service, scan geometry, speed scaling, and real-world noise required additional tuning.

2 Introduction

Autonomous exploration is a fundamental problem in mobile robotics, where a robot must navigate and map an unknown environment without prior knowledge. In this assignment, the goal is autonomous exploration and mapping of an unknown indoor environment using TurtleBot3. The robot builds a map using Cartographer SLAM and explores by repeatedly selecting frontier regions (boundaries between known free space and unknown space). As the SLAM map updates are relatively slow, exploration also requires a reactive obstacle avoidance layer driven directly by the laser scanner.

3 System Overview and architecture

The final system consists of three custom ROS2 nodes. The nodes communicate using standard ROS topics (OccupancyGrid, PoseStamped, Path, LaserScan and Twist). A simplified architecture is shown below with ROS2 topic flows between them.

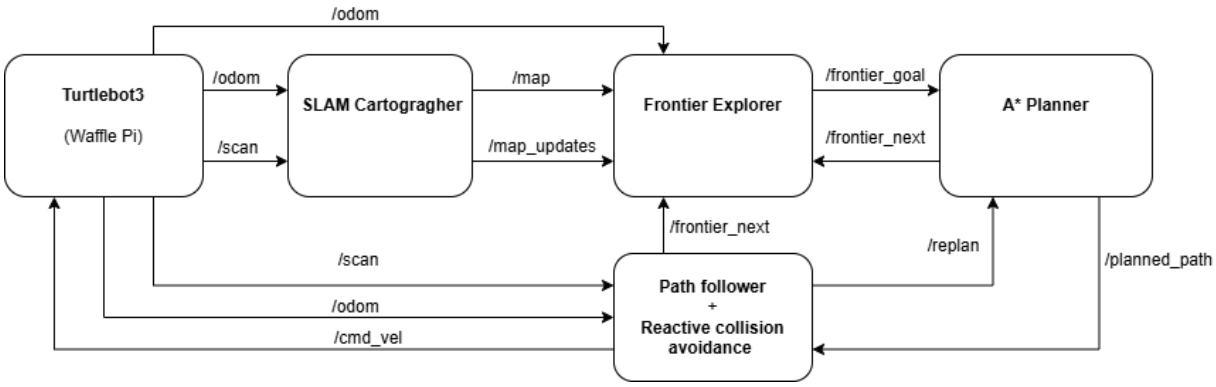


Figure 1: Simplified node-level architecture showing deliberative and reactive components..

3.1 Hybrid architecture

The controller is of a hybrid architecture: a deliberative layer reasons over the global occupancy grid to select exploration goals and plan paths, while a reactive layer continuously monitors laser

measurements to avoid collisions and recover from failures.

3.1.1 Deliberative components (planning / decision making)

The deliberative layer operates on a global map generated by the Cartographer SLAM algorithm and is responsible for:

1. **Frontier selection node:** Extracts frontiers from the occupancy grid, clusters the frontiers, ranks the goals, and publishes the next exploration goal.
2. **A*planner node:** Plans on the inflated occupancy grid and publishes a global path for the robot to follow to the selected exploration goal. It also supports re-planning on map updates and failure recovery.

This layer reasons at a higher level and makes long-term navigation decisions.

3.1.2 Reactive Components

While the deliberative layer plans long-term routes using a global map, the reactive layer handles fast, local responses based purely on what the sensors are currently perceiving, ensuring that the robot can react immediately to its surroundings. It operates directly on the laser scan data from the LiDAR, offering a fast sensor based control. The layer is responsible for:

Path follower + obstacle avoidance node: This node helps to track and follow the planned path, but if the laser scanner detects close obstacles, it switches into reactive avoidance behaviors by temporarily deviating from the global path. If the situation doesn't improve for a certain period it can also request re-planning or skip to the next frontier exploration goal accordingly.

This layer ensures safety and robustness despite delays or inaccuracies in the map.

3.2 Nodes and topics flow

The software was structured into modular ROS2 nodes and a simplified data flow is:

- **Cartographer SLAM** – publishes */map* and incremental */map_updates*.
- **Frontier Selector** – subscribes to /map + /map_updates, tracks the robot pose using */tf*, and publishes */frontier_goal*
- **A* planner** – subscribes to */map*, */map_updates*, */odom*, */frontier_goal*, and */replan*; publishes */planned_path* and may publish */frontier_next* when planning repeatedly fails.
- **path follower** – subscribes to */planned_path*, */scan*, */odom*, */frontier_goal*; publishes */cmd_vel* plus recovery signals */replan* and */frontier_next*

4 Algorithms

4.1 Frontier extraction and clustering

Frontiers are detected on the occupancy grid as free cells that are adjacent (8-neighborhood) to unknown cells. To reduce the risk of selecting goals too close to walls, occupied cells are inflated by a radius corresponding to the robot footprint plus a safety margin.

Frontier cells are grouped into clusters using an 8-connected flood-fill implemented with **Breadth-First Search (BFS)**. Starting from an unvisited frontier cell, BFS expands to all neighboring frontier cells (including diagonals), repeatedly adding newly found neighbors to a FIFO queue until no more connected frontier cells remain. All cells reached in this process form one connected cluster, then very small clusters are then discarded to avoid selecting noisy or unreliable frontiers.

For each remaining cluster, a single navigation target is chosen as a representative goal by selecting a cell close to the median row and median column of the cluster. This median-based choice is robust to irregular cluster shapes and tends to place the goal in the "central" part of the frontier. Finally, clusters are ranked to decide the next exploration goal, prioritizing shorter distance to the robot and larger cluster size, as larger clusters could typically correspond to more informative unexplored regions.

High-Level Frontier Exploration Algorithm

1. Subscribe to `/map` + `/map_updates`.
2. When map changes:
   (a) Build `free`, `unknown`, `occ` masks from occupancy values.
   (b) Inflate `occ` by `(robot_radius + margin)` → `inflated_occ`.
   (c) Compute `frontier_mask = free AND adjacent_to_unknown`.
   (d) Cluster `frontier_mask` using BFS connected components (8-neighborhood).
   (e) if the number of points in a cluster < `min_cluster_size`: omit the cluster.
   (f) For each cluster: compute representative goal (median-based) in world coordinates.
   (g) Rank clusters by `(distance_to_robot, -cluster_size)`.
3. If a new goal is requested OR current goal is invalid, AND cooldown period passed: publish `/frontier_goal` = top-ranked cluster representative in `map` frame.
4. If current goal reached OR `goal_exploration_timeout` passed: advance to next best ranked frontier cluster goal.
5. If no frontiers remain OR too many failed cycles: stop (exploration complete).

Algorithm 1 Frontier Exploration (Main Loop)

procedure FRONTIEREXPLORER
  $currentGoal \leftarrow$ None
  $selectedRank \leftarrow 0$, $cycleCount \leftarrow 0$
  while exploration is running **do**
    Get latest *map* and robot *pose* in `map` frame
    if *map* missing **or** *pose* unavailable **then**
      continue ▷ Cannot choose goals reliably
    end if
    $candidates \leftarrow$ BUILDFRONTIERCANDIDATES(map) ▷ see Algorithm 2
    if $candidates = \emptyset$ **then**
      stop ▷ No frontiers remain
    end if
    $ranked \leftarrow$ RANKCANDIDATES($candidates, pose$) ▷ see Algorithm 3
    if REACHED(pose, currentGoal) **then**
      $currentGoal \leftarrow$ None; $selectedRank \leftarrow selectedRank + 1$
      $cycleCount \leftarrow 0$ ▷ Succeeded; try next
    else if NEXTREQUESTED **then**
      $currentGoal \leftarrow$ None; $selectedRank \leftarrow selectedRank + 1$
      ▷ External request to move on
    else if GOALINVALID(currentGoal, map) **then**
      $currentGoal \leftarrow$ None ▷ Goal became unsafe/invalid
    end if
    if $currentGoal =$ None **and** cooldown elapsed **then**
      if $selectedRank \geq |ranked|$ **then**
        $selectedRank \leftarrow 0$; $cycleCount \leftarrow cycleCount + 1$
        if $cycleCount \geq maxFullCycles$ **then**
          stop ▷ Repeated full cycles with no progress
        end if
      end if
      $currentGoal \leftarrow ranked[selectedRank].goal$
      PUBLISH(currentGoal) ▷ Publish to `/frontier_goal`
    end if
  end while
end procedure

Algorithm 2 BUILD FRONTIER CANDIDATES

procedure BUILDFRONTIERCANDIDATES(map)
  Classify cells into *free*, *occupied*, *unknown*
  $r \leftarrow \lceil (\text{robotRadius} + \text{margin})/\text{resolution} \rceil$
  Inflate *occupied* by r cells ▷ Safety buffer
  $frontier \leftarrow$ free cells adjacent (8-neighbor) to unknown
  Cluster $frontier$ via BFS (8-connected components)
  Remove clusters with size $< minClusterSize$ ▷ Drop noise
  for all cluster C **do**
    $(r_{med}, c_{med}) \leftarrow$ median row/col of cells in C
    $p \leftarrow$ cell in C closest to (r_{med}, c_{med}) ▷ Medoid-like representative
    Convert p to world (x, y); add candidate $(goal = (x, y), size = |C|)$
  end for
  return candidates
end procedure

Algorithm 3 RANK FRONTIER CANDIDATES

procedure RANKCANDIDATES(candidates, pose)
  for all candidate i **do**
    $d_i \leftarrow$ distance(robot, $goal_i$)
  end for
  Sort by $(d_i \uparrow,\ size_i \downarrow)$ ▷ Closer first, break ties by bigger cluster
  return ranked list
end procedure

4.2 Path Planning (A*)

The global planner computes a path from the current robot pose to the current frontier exploration goal using A* search on a 2D grid. Before planning, the map is converted into a binary occupancy mask and inflated by robots radius plus a small margin to provide clearance. As the frontier exploration goals could lie close to unknown or inflated regions, which would disrupt the path planning, multiple nearby free goal candidates within a search radius to the goal cell are generated and tries several goal candidates in order of increasing distance from the requested goal.

The planner uses 8-connected motion (cost 1 for cardinal moves and sqrt(2) for diagonals) and a Euclidean heuristic. If the goal is still unreachable and no complete path exists, the implementation would still return a best-effort partial path to the expanded node closest to the goal.

The path planner node also listens to */map_updates* and can re-plan automatically on updates but the rate is limited. The planner also offers re-planning when reactive collision avoidance layer fails. Post the path generation from A* algorithm and before planned path is published as a navigation message on the topic */planned_path*, two smoothing steps are used:

1. **shortcut / string-pulling** : It removes unnecessary intermediate points if line-of-sight is free (Bresenham grid traversal).

2. **Moving average smoothing** : It then applies a moving average filter on the resulting (x,y) way-points.

Algorithm 4 A* Planner Node: event handling (`/frontier_goal`, `/replan`, `/map_updates`)

Require: Occupancy grid stream via `/map`, `/map_updates`
Require: Robot pose from `/odom`, exploration goal from `/frontier_goal`
Require: Optional replan trigger via `/replan`
Require: Parameters: $replanCooldown$
Ensure: Publish `/planned_path`; optionally publish `/frontier_next`

Subscribe to `/frontier_goal`, `/map`, `/map_updates`, `/odom`, `/replan`
$currentGoalWorld \leftarrow \emptyset$
$lastPlanTime \leftarrow -\infty$
while ROS is running **do**
  if `/frontier_goal` received **then**
    $currentGoalWorld \leftarrow$ received goal
    PlanToGoal($currentGoalWorld$) ▷ see Algorithm 5
  end if
  if `/replan` received **then** ▷ Reactive layer requests replanning (e.g., blocked/stuck)
    if $currentGoalWorld \neq \emptyset$ **and** $(now - lastPlanTime) \geq replanCooldown$ **then**
      PlanToGoal($currentGoalWorld$)
    end if
  end if
  if `/map_updates` received **and** replanning enabled **and** rate-limit allows **then** ▷ Optional: keep path consistent with new map data
    if $currentGoalWorld \neq \emptyset$ **then**
      PlanToGoal($currentGoalWorld$)
    end if
  end if
end while

Algorithm 5 PlanToGoal: High-level planning pipeline

Require: Occupancy grid $grid$, robot pose from `/odom`, goal $goalWorld$

Require: Parameters: r_{infl}, $goalSearchRadius$, $startSearchRadius$, $goalCandidateAttempts$, $maxExpansions$

Ensure: Publish `/planned_path`; optionally `/frontier_next`

$(x_r, y_r) \leftarrow$ pose from `/odom`
$start \leftarrow$ WorldToGrid$((x_r, y_r))$, $goal \leftarrow$ WorldToGrid$(goalWorld)$
$occ \leftarrow$ OccupiedMask$(grid)$
$blocked \leftarrow$ Inflate(occ, r_{infl})
$start \leftarrow$ FixStartIfBlocked$(start, blocked, startSearchRadius)$
$G \leftarrow$ FixGoalIfBlocked$(goal, blocked, goalSearchRadius)$ ▷ G = goal candidates
$N_g \leftarrow goalCandidateAttempts$
$N_e \leftarrow maxExpansions$
$(selected, success) \leftarrow$ SearchBestPath$(start, G, blocked, N_g, N_e)$ ▷ see Algorithm 6
if not *success* **then**
    return ▷ no path (even partial) found
end if
if IsPartial$(selected)$ **then**
    UpdateFailureCounters
    if RepeatedFailures **then**
        publish `/frontier_next`
    end if
end if
$selected \leftarrow$ ShortcutSmoothing$(selected, blocked)$
$selected \leftarrow$ MovingAverageSmoothing$(selected)$
publish `/planned_path` $\leftarrow$ GridPathToWorldPath$(selected)$

Algorithm 6 SEARCHBESTPATH: A* over goal candidates with best-effort partial path

Require: *start*, set of goal candidates G, obstacle grid *blocked*
Require: Limits: $goalCandidateAttempts$, $maxExpansions$
Ensure: $(selectedPath, success)$ where $selectedPath$ may be full or partial

$bestFull \leftarrow \emptyset$
$bestPartial \leftarrow \emptyset, \quad bestScore \leftarrow +\infty$
for $i \leftarrow 1$ to $\min(goalCandidateAttempts, |G|)$ **do**
  $g \leftarrow G[i]$
  $(path, reached, score) \leftarrow \text{ASTAR}(start, g, blocked, maxExpansions)$
  if *reached* **then**
    $bestFull \leftarrow path$
    break
  else if $score < bestScore$ **then**
    $bestScore \leftarrow score$
    $bestPartial \leftarrow path$
  end if
end for
if $bestFull \neq \emptyset$ **then**
  return $(bestFull, \textbf{true})$
else if $bestPartial \neq \emptyset$ **then**
  return $(bestPartial, \textbf{true})$ ▷ best-effort partial
else
  return $(\emptyset, \textbf{false})$
end if

4.3 Path Follower and collision avoidance

The path follower executes */planned_path* as a sequence of way-points while continuously checking the latest LaserScan from the LiDAR. If the front sector is closer than a safety distance, a reactive avoidance state machine takes over: it chooses a turn direction based on side clearances, turns until the path is reachable again, and then drives forward while searching for a waypoint to rejoin. In parallel, the controller monitors progress to the frontier goal. If no progress is observed for a stall timeout, if the robot fails to recover from the reactive avoidance state machine, then it triggers replanning (*/replan*) to the path planner node. If several re-plans fail or if the robot remains stuck or exceeds a goal deadline, it requests the next frontier (*/frontier_next*) directly to the Frontier explorer node.

5 Results

The proposed algorithms were tested in both virtual (Gazebo) and real environment.

5.1 Simulation

The implemented system successfully performed autonomous exploration and mapping in the TurtleBot3 house world using the hybrid control architecture.

The frontier module consistently detected multiple frontier clusters and visualized them as colored groups. minimum points allowed in a cluster was set to 10 and maximum was set to 40. **Figure 2** shows a scenario in the virtual setup plotting the clusters and its centroids, the current pose of the robot and the planned path to explore the frontiers.

The A* planner produced feasible paths from the TurtleBot's current pose to the selected frontier goal. These paths were published and visualized as continuous trajectories in RViz, and the robot followed them through the environment. The update of the occupancy grid the in the simulation felt sluggish but the robot explored the environment smoothly without any manual intervention.

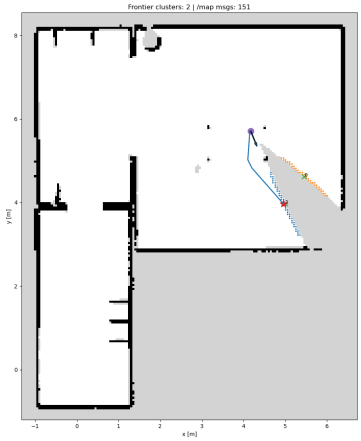


Figure 2: Simulation result - showing the TurtleBot's pose, clusters and the planned path.

5.2 Real Robot Deployment

After adapting the system to the physical **TurtleBot3 Waffle Pi** controlled by a Raspberry Pi4 and uses a 360 Laser Distance Sensor LDS-02 for mapping the environment, the robot was able to explore and expand the map of the lab environment using the same overall pipeline.

Though the LiDAR values are nosier compared to that of simulation, with the physical bot the update of the map felt quite faster than that of the simulation. The Lab environment had reflective or shinny metals and transparent glasses which created uncertainty in the LiDAR readings and hence the obtained real world map collapses at times. **Figure 3** shows the partly mapped Lab environment, WaffleBot's pose and heading, clusters and its centroid, and the feasible frontier goal to explore and the obtained global path, In the specific scenario the algorithm choose to explore the nearest cluster.

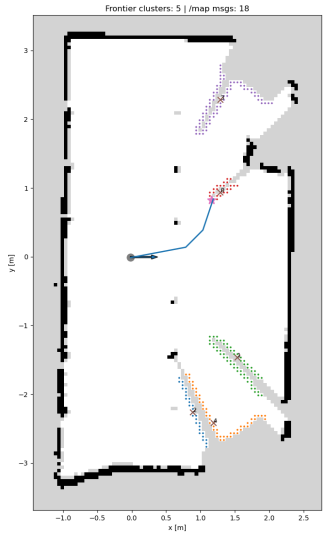


Figure 3: Real world result - showing the TurtleBot's pose with heading, clusters and the planned path.

6 Development experience and Challenges

Compared to the simulation the real robot moved significantly faster and the sensor noise to the surrounding was more prominent. Also in the actual robot localisation drift occurred more frequently.

Having three different nodes for unique tasks (frontier exploration, A* path planner and the path follower with collision avoidance) boosted the performance significantly. For example, the A* global path planner node was publishing new optimal paths faster and the reactive layer was well responsive to obstacles.

The main practical challenge was balancing aggressive exploration with safety and robustness. Early versions frequently selected goals too close to obstacles or in partially known areas, which caused oscillations and repeated re-plans. The following changes improved stability:

- **obstacle inflation** with a footprint-based radius and a margin helped for both frontier goal detection and A* path planning.
- **Minimum frontier cluster size filtering** helped in omitting the noisy frontier points within the already explored region.
- **Maximum Command velocity** - limiting the maximum commond velocity for both linear and angular movement assisted in smooth robot movement and reducing drift.
- **Medoids and multi-candidate goal selection in A*** - Considering the medoids of the clusters than the mean or the median point made sure that exploration goal is not within the unexplored region. In addition, the multi-candidate goal selection within a selected radius to the goal also provided robust behaviour in finding and publishing global path consistently.

- **Path smoothing** made sure the global path was not that jaggy and assisted with smooth robot velocity.
- **Reactive layer** with a proper safety margin distance assisted with collision avoidance by temporarily deviating the robot from the path and then rejoining it.

7 Conclusion

A hybrid exploration system was implemented and demonstrated for TurtleBot3 in simulation and on a physical robot. Frontier clustering enabled informative goal selection, and A* planning on an inflated map produced safe global routes. The reactive layer handled map latency and local dead-ends by avoiding obstacles and triggering re-plans when needed. This work also demonstrates the importance of layered control architecture in real-word. Future work includes improved frontier scoring (information gain vs cost) and smoother local control.